

Multi-Cloud Resource Management with Terraform, Kubernetes, and AWS S3

Introduction

As organizations increasingly migrate to cloud environments, managing these resources becomes more complex. Automation tools like Terraform provide the ability to define infrastructure as code, leading to more efficient and reliable deployments. The implementation of a multi-cloud resource management solution using Terraform to automate the provisioning of an AWS S3 bucket and a Kubernetes cluster on AWS. It describes the steps taken to deploy a sample Node.js application on the Kubernetes cluster, highlighting the advantages of using Infrastructure as Code (IaC) practices for efficient cloud resource management.

Concepts Used

- **Terraform:** An open-source tool that enables users to define and provision infrastructure using a declarative configuration language. It allows for the automation of infrastructure management across multiple cloud providers.
- **Kubernetes:** An open-source container orchestration platform that automates the deployment, scaling, and management of containerized applications, providing a robust framework for running applications in a cloud environment.
- **AWS S3:** Amazon Simple Storage Service (S3) is an object storage service that offers high durability, availability, and scalability for storing and retrieving any amount of data from anywhere on the web.

Problem Statement

The primary goal was to create and manage multi-cloud resources using Terraform. This included:

- Provisioning an AWS S3 bucket for data storage.
- Setting up a Kubernetes cluster on AWS using Elastic Kubernetes Service (EKS).
- Deploying a simple Node.js application on the Kubernetes cluster to validate the setup.

Implementation Steps

1. Environment Setup

Before starting, it was crucial to ensure that all necessary tools were installed and configured. The following commands were executed to check the installed versions of each tool:

```
aws --version
```

```
D:\>aws --version  
aws-cli/2.18.12 Python/3.12.6 Windows/10 exe/AMD64
```

```
terraform --version
```

```
D:\>terraform --version  
Terraform v1.9.8  
on windows_amd64
```

```
kubect1 version --client
```

```
D:\>kubect1 version --client  
Client Version: v1.31.0  
Kustomize Version: v5.4.2
```

```
python --version
```

```
D:\>python --version  
Python 3.12.1
```

```
node --version
```

```
D:\>node --version  
v20.10.0
```

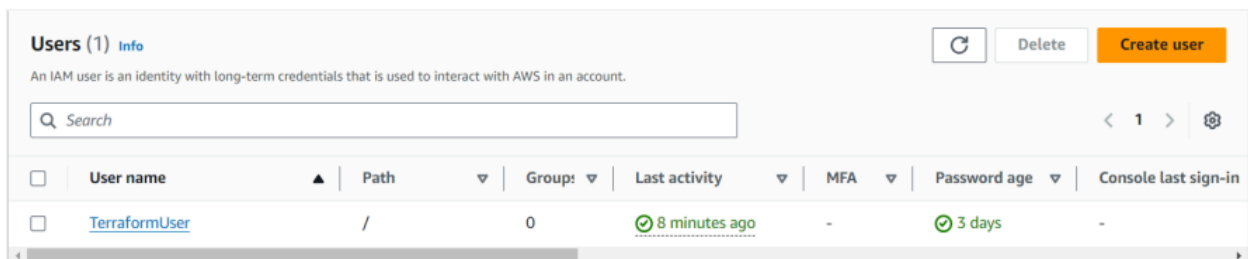
```
git --version
```

```
D:\>git --version  
git version 2.42.0.windows.2
```

2. Create IAM User Role

A new IAM user role named TerraformUser was created in the AWS Management Console. This role grants necessary permissions to manage AWS resources:

- **Policy Attachments:**
 - AmazonS3FullAccess: Full access to S3 resources.
 - AmazonEKSClusterPolicy: Permissions to manage EKS clusters.
 - AmazonEKSServicePolicy: Permissions for EKS service operations.
 - AmazonEKSWorkerNodePolicy: Permissions for EKS worker nodes.



☐	User name	Path	Group	Last activity	MFA	Password age	Console last sign-in
☐	TerraformUser	/	0	8 minutes ago	-	3 days	-

3. Create Project Directory

A dedicated directory for the Terraform project was created to organize all related files. This directory serves as the workspace for managing the infrastructure.

```
mkdir my-terraform-project  
cd my-terraform-project
```

```
D:\AdvDevOps>mkdir my-terraform-project
```

```
D:\AdvDevOps>cd my-terraform-project
```

4. Write Terraform Script

The core of the implementation is the main.tf file, which defines the infrastructure components. This file includes configuration for the S3 bucket and EKS cluster.

```
provider "aws" {
  region      = "us-west-2" # Specify the AWS region
  access_key  = "YOUR_ACCESS_KEY" # AWS Access Key
  secret_key  = "YOUR_SECRET_KEY" # AWS Secret Key
}

resource "aws_s3_bucket" "my_bucket" {
  bucket = "my-unique-bucket-name" # Unique bucket name
  acl    = "private" # Access control list
}

resource "aws_eks_cluster" "my_cluster" {
  name      = "my-eks-cluster" # Name of the EKS cluster
  role_arn  = "arn:aws:iam::YOUR_ACCOUNT_ID:role/TerraformUser" # Role
  for EKS
  vpc_config {
    subnet_ids = ["subnet-xxxxxxx", "subnet-yyyyyyy"] # Specify the
    subnet IDs
  }
}

output "s3_bucket_id" {
  value = aws_s3_bucket.my_bucket.id # Output the S3 bucket ID
}

output "eks_cluster_name" {
  value = aws_eks_cluster.my_cluster.name # Output the EKS cluster
  name
}
```

```
D:\AdvDevOps\my-terraform-project>echo. > main.tf
```

```
main.tf
1  provider "aws" {
2      region = "us-east-1"
3  }
4
5  # Create an S3 bucket
6  resource "aws_s3_bucket" "saachi_bucket" {
7      bucket = "saachi-unique-bucket-name" # Use a globally unique name
8  }
9
10 # Create a VPC
11 resource "aws_vpc" "saachi_vpc" {
12     cidr_block = "10.0.0.0/16"
13 }
14
15 # Create subnets in multiple availability zones
16 resource "aws_subnet" "saachi_subnet" {
17     count                = 2
18     vpc_id               = aws_vpc.saachi_vpc.id
19     cidr_block           = cidrsubnet(aws_vpc.saachi_vpc.cidr_block, 8, count.
20     index)
21     availability_zone     = element(data.aws_availability_zones.available.names,
22     count.index)
23 }
```

```
main.tf
23 data "aws_availability_zones" "available" {}
24
25 # Create an EKS cluster
26 resource "aws_eks_cluster" "saachi_cluster" {
27     name          = "saachi-cluster"
28     role_arn      = "arn:aws:iam::692489837678:role/lab-role" # Use your existing lab
29     role
30     vpc_config {
31         subnet_ids = aws_subnet.saachi_subnet[*].id
32     }
33 }
34
35 # Create an EKS Node Group
36 resource "aws_eks_node_group" "saachi_node_group" {
37     cluster_name    = aws_eks_cluster.saachi_cluster.name
38     node_group_name = "saachi-node-group"
39     node_role_arn   = "arn:aws:iam::692489837678:role/lab-role" # Use the same lab
40     role
41     subnet_ids      = aws_subnet.saachi_subnet[*].id
42
43     scaling_config {
```

```

44     desired_size = 2
45     max_size     = 3
46     min_size     = 1
47 }
48 }
49
50 # Output the S3 bucket name
51 ✓ output "s3_bucket_name" {
52     value = aws_s3_bucket.saachi_bucket.bucket
53 }
54
55 # Output the EKS cluster name
56 ✓ output "eks_cluster_name" {
57     value = aws_eks_cluster.saachi_cluster.name
58 }

```

5. Configure AWS Credentials

AWS credentials must be configured to allow Terraform to authenticate and manage resources. The environment variables were set as follows:

```
export AWS_ACCESS_KEY_ID="YOUR_ACCESS_KEY"
export AWS_SECRET_ACCESS_KEY="YOUR_SECRET_KEY"
export AWS_SESSION_TOKEN="YOUR_SESSION_TOKEN"
```

```
D:\AdvDevOps\my-terraform-project>set AWS_ACCESS_KEY_ID=ASIA2C044LRXNW3YC3RX
```

```
D:\AdvDevOps\my-terraform-project>set AWS_SECRET_ACCESS_KEY=EpKf0uRMswZK+q31YAp7SACRsxgHps6772prg0Xd
```

```
D:\AdvDevOps\my-terraform-project>set AWS_SESSION_TOKEN=IQo3b3ZpZj21X2UwZjFEMhAKXVdXdLdc3QtHjHMEUCQCE3F/sPZvFtdjPLWQV4N+3jJmSSq67AUMhG5Ls+0w+i9jE1gAVXzMs3sqsq8YXCMQK33497UBwKMLFv0vAq/zpq0wV1Pp//////////ARAAQwG2T1E00K4dc2nZp1DePoAww3vYFLXdc1q1tALNRW6t3qA/YCQ2m3dPmK0h0o9y2z3q1vHqsFCRE0TjP8FhmYpWzL2E4L1K2GfH1XrLbUvFb1JLzK9u29yASv6V1ChYk7ZnPMRvUw2Zbtz8vY295vAFR5SLnU1tYp1k1uW6P5es/qH2j3p1d85yG6o1Qhnpmlcpobv9CyPqA8U9vSE03XhV1v+63Z63oQvqkqPXBurdJ4F13rowh1r4AE5DScuMfG8b1h1RbF6J3xwpu555pgk00u8c8VUw1vZnV2wM1Tt6gTEgEdxmZgk1JmW3pZJ1Rj9r15u5F1RvY651K2jE2WvR7G5cZnG5C1N2F7X2Xk0p1L9tFvFYZeXGm74go0pB1JL2M1Td+zv56gSv8hbw/Lr/c+eN0b5M1X0bT308j1nL1IozS5zREfhw351v6bV5k44P21D1E6A5agp5buz29wJw1u1RHWK98J2Tjv1gusYR8KB6755P4oC7u/bK2cuhvF7/CEU2B0K1EstrvXN0M492c23q1Jd29dRPPv+1B9NMU24u61as1Z+eN0b55M6w
```

6. Initialize Terraform

Before applying the configuration, Terraform needs to be initialized to download the necessary providers and set up the environment. The following commands were executed:

```
terraform init    # Initializes the Terraform configuration
```

```
D:\AdvDevOps\my-terraform-project>terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v5.72.1...
- Installed hashicorp/aws v5.72.1 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.
```

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

terraform plan # Displays the execution plan

```
D:\AdvDevOps\my-terraform-project>terraform plan
data.aws_availability_zones.available: Reading...
data.aws_availability_zones.available: Read complete after 1s [id=us-east-1]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_eks_cluster.saachi_cluster will be created
+ resource "aws_eks_cluster" "saachi_cluster" {
  + arn = (known after apply)
  + bootstrap_self_managed_addons = true
  + certificate_authority = (known after apply)
  + cluster_id = (known after apply)
  + created_at = (known after apply)
  + endpoint = (known after apply)
  + id = (known after apply)
  + identity = (known after apply)
  + name = "saachi-cluster"
  + platform_version = (known after apply)
  + role_arn = "arn:aws:iam::692489837678:role/lab-role"
  + status = (known after apply)
  + tags_all = (known after apply)
  + version = (known after apply)

  + access_config (known after apply)

  + kubernetes_network_config (known after apply)

  + upgrade_policy (known after apply)

  + vpc_config {
    + cluster_security_group_id = (known after apply)
    + endpoint_private_access = false
    + endpoint_public_access = true
    + public_access_cidrs = (known after apply)
    + subnet_ids = (known after apply)
  }
}
```

```

+ ipv6_native                = false
+ map_public_ip_on_launch    = false
+ owner_id                   = (known after apply)
+ private_dns_hostname_type_on_launch = (known after apply)
+ tags_all                   = (known after apply)
+ vpc_id                     = (known after apply)
}

# aws_vpc.saachi_vpc will be created
+ resource "aws_vpc" "saachi_vpc" {
+   arn                = (known after apply)
+   cidr_block         = "10.0.0.0/16"
+   default_network_acl_id = (known after apply)
+   default_route_table_id = (known after apply)
+   default_security_group_id = (known after apply)
+   dhcp_options_id     = (known after apply)
+   enable_dns_hostnames = (known after apply)
+   enable_dns_support   = true
+   enable_network_address_usage_metrics = (known after apply)
+   id                  = (known after apply)
+   instance_tenancy    = "default"
+   ipv6_association_id = (known after apply)
+   ipv6_cidr_block     = (known after apply)
+   ipv6_cidr_block_network_border_group = (known after apply)
+   main_route_table_id = (known after apply)
+   owner_id            = (known after apply)
+   tags_all            = (known after apply)
}

```

Plan: 6 to add, 0 to change, 0 to destroy.

Changes to Outputs:

```

+ eks_cluster_name = "saachi-cluster"
+ s3_bucket_name   = "saachi-unique-bucket-name"

```

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

terraform apply

```

D:\AdvDevOps\my-terraform-project>terraform apply
data.aws_availability_zones.available: Reading...
data.aws_availability_zones.available: Read complete after 1s [id=us-east-1]

```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

```

# aws_eks_cluster.saachi_cluster will be created
+ resource "aws_eks_cluster" "saachi_cluster" {
+   arn                = (known after apply)
+   bootstrap_self_managed_addons = true
+   certificate_authority = (known after apply)
+   cluster_id          = (known after apply)
+   created_at           = (known after apply)
+   endpoint              = (known after apply)
+   id                   = (known after apply)
+   identity              = (known after apply)
+   name                 = "saachi-cluster"
+   platform_version     = (known after apply)
+   role_arn              = "arn:aws:iam::692489837678:role/lab-role"
+   status                = (known after apply)
+   tags_all              = (known after apply)
+   version               = (known after apply)

+   access_config (known after apply)

+   kubernetes_network_config (known after apply)

+   upgrade_policy (known after apply)

+   vpc_config {
+     cluster_security_group_id = (known after apply)
+     endpoint_private_access    = false
+     endpoint_public_access    = true
+     public_access_cidrs       = (known after apply)
+     subnet_ids                = (known after apply)
+     vpc_id                    = (known after apply)

```

```
}

Plan: 0 to add, 0 to change, 3 to destroy.

Changes to Outputs:
  - eks_cluster_name = "saachi-cluster" -> null

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_subnet.saachi_subnet[0]: Destroying... [id=subnet-0d7c00eca42f9077d]
aws_subnet.saachi_subnet[1]: Destroying... [id=subnet-04fd6058891597a0c]
aws_subnet.saachi_subnet[1]: Destruction complete after 2s
aws_subnet.saachi_subnet[0]: Destruction complete after 2s
aws_vpc.saachi_vpc: Destroying... [id=vpc-03b72a34c9508475d]
aws_vpc.saachi_vpc: Destruction complete after 1s

Apply complete! Resources: 0 added, 0 changed, 3 destroyed.

Outputs:

s3_bucket_name = "saachi-unique-bucket-name"
```

7. Verify S3 Bucket Creation

After applying the Terraform configuration, the creation of the S3 bucket was verified using the AWS CLI command:

```
aws s3 ls # Lists all S3 buckets in the account
```

```
D:\AdvDevOps\my-terraform-project>aws s3 ls
2024-10-23 17:47:36 saachi-unique-bucket-name
```

```
D:\AdvDevOps\my-terraform-project>aws s3 ls s3://saachi-unique-bucket-name/
2024-10-23 17:55:48      15905 javascript.docx
```

The screenshot shows the Amazon S3 console interface. At the top, there's a navigation bar with 'Amazon S3' and 'Buckets'. Below this, a banner for 'Account snapshot' is visible. The main section is titled 'General purpose buckets' and shows a list of buckets. A single bucket, 'saachi-unique-bucket-name', is listed with the following details:

Name	AWS Region	IAM Access Analyzer	Creation date
saachi-unique-bucket-name	US East (N. Virginia) us-east-1	View analyzer for us-east-1	October 23, 2024, 19:16:43 (UTC+05:30)

Buttons for 'Copy ARN', 'Empty', 'Delete', and 'Create bucket' are visible above the table. A search bar and pagination controls are also present.

8. Create Application Deployment Files

Deployment files for the Kubernetes application were created, including deployment.yaml and service.yaml. These files define how the application runs in the cluster and how it is accessed.

Deployment File (deployment.yaml):

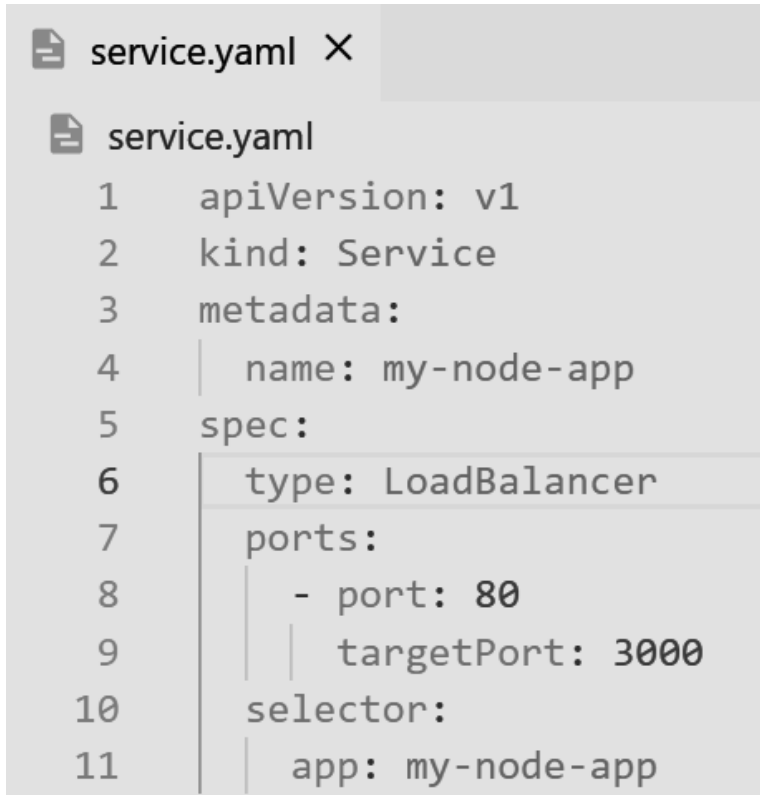
```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-node-app
spec:
  replicas: 2 # Number of replicas
  selector:
    matchLabels:
      app: my-node-app
  template:
    metadata:
      labels:
        app: my-node-app
    spec:
      containers:
        - name: my-node-app
          image: my-node-app-image:latest # Image of the application
          ports:
            - containerPort: 3000
```



```
deployment.yaml X
deployment.yaml
2  kind: Deployment
3  metadata:
4    name: my-node-app
5  spec:
6    replicas: 2
7    selector:
8      matchLabels:
9        app: my-node-app
10   template:
11     metadata:
12       labels:
13         app: my-node-app
14     spec:
15       containers:
16         - name: my-node-app
17           image: your-dockerhub-username/my-node-app
18           ports:
19             - containerPort: 3000
```

Service File (service.yaml):

```
apiVersion: v1
kind: Service
metadata:
  name: my-node-app
spec:
  type: LoadBalancer # Exposes the service to the internet
  ports:
    - port: 80
      targetPort: 3000
  selector:
    app: my-node-app
```

A screenshot of a code editor window titled 'service.yaml'. The editor shows the following YAML configuration for a Kubernetes Service:

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    | name: my-node-app
5  spec:
6    | type: LoadBalancer
7    | ports:
8    |   - port: 80
9    |     | targetPort: 3000
10   selector:
11     | app: my-node-app
```

9. Create Application Deployment Files

Deployment files for the Kubernetes application were created, including deployment.yaml, service.yaml, a Dockerfile, and app.js. These files define how the application runs in the cluster, how it is accessed, and how it is packaged.

a. Dockerfile

The Dockerfile specifies how to build the Docker image for the Node.js application.

Content of Dockerfile:

```
dockerfile
Copy code
# Use the official Node.js image as the base image
FROM node:14

# Set the working directory inside the container
```

```
WORKDIR /usr/src/app

# Copy package.json and package-lock.json to the working directory
COPY package*.json ./

# Install dependencies
RUN npm install

# Copy the application source code to the container
COPY . .

# Expose the application port
EXPOSE 3000

# Command to run the application
CMD ["node", "app.js"]
```

 **dockerfile**

```
1  # Use the official Node.js image as a base
2  FROM node:14
3
4  # Set the working directory
5  WORKDIR /usr/src/app
6
7  # Copy package.json and package-lock.json
8  COPY package*.json ./
9
10 # Install dependencies
11 RUN npm install
12
13 # Copy the rest of the application code
14 COPY . .
15
16 # Expose the port the app runs on
17 EXPOSE 3000
18
19 # Command to run the application
20 CMD ["node", "app.js"]
21
```

b. Node.js Application

The app.js file contains a simple Node.js application that listens for HTTP requests.

Content of app.js:

```
javascript
Copy code
const express = require('express');
const app = express();
const port = 3000;
// Simple endpoint to respond with a message
app.get('/', (req, res) => {
  res.send('Hello, World! This is my Node.js application running on
Kubernetes!');
});
// Start the server
app.listen(port, () => {
  console.log(`App listening at http://localhost:${port}`);
});
```

```
1  const express = require('express');
2  const app = express();
3  const port = 3000;
4
5  app.get('/', (req, res) => {
6    res.send('Hello, World from Saachi Raheja!');
7  });
8
9  app.listen(port, () => {
10    console.log(`Server running at http://localhost:${port}`);
11  });
```

c. Package Management

A package.json file is needed to manage the application dependencies. It can be created using the following command:

npm init -y # Initializes a new Node.js project and creates package.json

```
D:\my-node-app>npm init -y
Wrote to D:\my-node-app\package.json:

{
  "name": "my-node-app",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

The express package is added as a dependency:

npm install express # Installs the Express framework

```
D:\my-node-app>npm install express

added 65 packages, and audited 66 packages in 5s

13 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

10. Deploy the Application to Kubernetes

The application was deployed to the Kubernetes cluster using the kubectl command. First, the kubeconfig was updated to allow access to the EKS cluster:

```
D:\>minikube start
* minikube v1.34.0 on Microsoft Windows 10 Pro 10.0.19045.5011 Build 19045.5011
* Automatically selected the docker driver
* Using Docker Desktop driver with root privileges
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.45 ...
* Creating docker container (CPUs=2, Memory=4000MB) ...
! Failing to connect to https://registry.k8s.io/ from inside the minikube container
* To pull new external images, you may need to configure a proxy: https://minikube.sigs.k8s.io/docs/reference/networking/proxy/
* Preparing Kubernetes v1.31.0 on Docker 27.2.0 ...
  - Generating certificates and keys ...
  - Booting up control plane ...
  - Configuring RBAC rules ...
* Configuring bridge CNI (Container Networking Interface) ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass
* kubectl not found. If you need it, try: 'minikube kubectl -- get pods -A'
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

```
D:\>minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

```
aws eks --region us-west-2 update-kubeconfig --name my-eks-cluster
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
```

11. Verification of Deployment

The status of the Kubernetes pods and services was checked to ensure that the application was running successfully:

```
kubectl get pods # Lists all pods in the current namespace
kubectl get services # Lists all services in the current namespace
```

12. Clean Up

To avoid incurring costs for resources that are no longer needed, the created resources were destroyed using Terraform:

```
terraform destroy # Destroys all resources managed by Terraform
```

```
D:\AdvDevOps\my-terraform-project>terraform destroy
data.aws_availability_zones.available: Reading...
aws_vpc.saachi_vpc: Refreshing state... [id=vpc-02f7f281d0ede2c45]
aws_s3_bucket.saachi_bucket: Refreshing state... [id=saachi-unique-bucket-name]
data.aws_availability_zones.available: Read complete after 1s [id=us-east-1]
aws_subnet.saachi_subnet[0]: Refreshing state... [id=subnet-01b1583142a8473ca]
aws_subnet.saachi_subnet[1]: Refreshing state... [id=subnet-0f40f9f6a108e645f]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
- destroy

Terraform will perform the following actions:

# aws_s3_bucket.saachi_bucket will be destroyed
- resource "aws_s3_bucket" "saachi_bucket" {
  - arn                = "arn:aws:s3:::saachi-unique-bucket-name" -> null
  - bucket             = "saachi-unique-bucket-name" -> null
  - bucket_domain_name = "saachi-unique-bucket-name.s3.amazonaws.com" -> null
  - bucket_regional_domain_name = "saachi-unique-bucket-name.s3.us-east-1.amazonaws.com" -> null
  - force_destroy      = false -> null
  - hosted_zone_id     = "Z3AQBSTGFYJSTF" -> null
  - id                = "saachi-unique-bucket-name" -> null
  - object_lock_enabled = false -> null
  - region            = "us-east-1" -> null
  - request_payer      = "BucketOwner" -> null
  - tags               = {} -> null
  - tags_all           = {} -> null
  # (3 unchanged attributes hidden)
```

Here we face an issue to destroy the bucket

```
Error: deleting S3 Bucket (saachi-unique-bucket-name): operation error S3: DeleteBucket, https response error StatusCode: 409, RequestID: CGNXD2C42Y3Q0N7T, HostID: RzwN1ibqyvFoQ/R1y0L1WjxqoNKj0nQaaUur6ef85rr89cnFCUDEOLG90o38ZgDs0zn7skfGQY=, api error BucketNotEmpty: The bucket you tried to delete is not empty
```

So we run the following command:

```
D:\AdvDevOps\my-terraform-project>aws s3 rm s3://saachi-unique-bucket-name --recursive
delete: s3://saachi-unique-bucket-name/javascript.docx

D:\AdvDevOps\my-terraform-project>
D:\AdvDevOps\my-terraform-project>aws s3api delete-bucket --bucket saachi-unique-bucket-name
```

And then destroy it again :

```
D:\AdvDevOps\my-terraform-project>terraform destroy
data.aws_availability_zones.available: Reading...
aws_s3_bucket.saachi_bucket: Refreshing state... [id=saachi-unique-bucket-name]
data.aws_availability_zones.available: Read complete after 1s [id=us-east-1]
```

Changes to Outputs:

```
- eks_cluster_name = "saachi-cluster" -> null
- s3_bucket_name   = "saachi-unique-bucket-name" -> null
```

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Do you really want to destroy all resources?

Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

Destroy complete! Resources: 0 destroyed.

Key Feature & Application

Key Feature

Infrastructure as Code (IaC): The primary advantage of using Terraform is the ability to manage and provision infrastructure through code. This approach promotes:

- **Consistency:** Ensures that environments are provisioned the same way every time.
- **Version Control:** Allows changes to be tracked and managed through source control systems.
- **Automation:** Reduces manual intervention, minimizing human errors.

Application

1. **Multi-Cloud Resource Management:** Enables organizations to manage and orchestrate resources across multiple cloud environments using a unified toolset.
2. **Efficient Application Deployment:** Kubernetes facilitates the deployment, scaling, and management of containerized applications, ensuring they are resilient and available.
3. **Collaboration and Transparency:** Version-controlled infrastructure files promote collaboration among team members, enabling easier audits and modifications.
4. **Cost Management:** Automated provisioning and de-provisioning help optimize resource usage, reducing unnecessary costs.
5. **Security and Compliance:** Consistent application of security policies ensures compliance with organizational standards.

Conclusion

This overview illustrates the successful implementation of a multi-cloud resource management solution using Terraform, Kubernetes, and AWS S3. By following the structured approach outlined, organizations can effectively manage their cloud resources, leveraging modern DevOps practices to enhance operational efficiency and reliability.